Appendix for CuriousBot

Anonymous Authors

I. Method

A. IoU Computation

To compute the IoU of two point clouds, we first compute the distance between each point from two point clouds. If a point's minimum distance to another point cloud is smaller than the predefined threshold τ, this point is considered an "intersection". After counting all intersections, we can compute IoU by dividing the intersections by the total point number. The detailed algorithm is shown in Algorithm 1.

Algorithm 1 IoU Between Point Clouds

Input: Two point clouds $A \in \mathbb{R}^{N\times 3}$, $B \in \mathbb{R}^{M\times 3}$; threshold τ
Output: IoU matrix IoU

1: **if** $|A| = 0$ or $|B| = 0$ **then return** 0

2: Compute distances D between all points in A and B:

$$D_{k,l} = \|A[k] - B[l]\|, \quad \forall k = 1,\ldots,N; \quad l = 1,\ldots,M$$

3: Compute minimum distances for each point:

$$d_k^{(A)} = \min_l D_{k,l}, \quad \forall k = 1,\ldots,N$$

$$d_l^{(B)} = \min_k D_{k,l}, \quad \forall l = 1,\ldots,M$$

4: Create masks based on threshold τ:

$$\text{mask}_A[k] = \begin{cases} 1, & \text{if } d_k^{(A)} < \tau \\ 0, & \text{otherwise} \end{cases}$$

$$\text{mask}_B[l] = \begin{cases} 1, & \text{if } d_l^{(B)} < \tau \\ 0, & \text{otherwise} \end{cases}$$

5: Compute sums of masks:

$$S_A = \sum_{k=1}^{N} \text{mask}_A[k], \quad S_B = \sum_{l=1}^{M} \text{mask}_B[l]$$

6: Compute IoU:

$$\text{IoU} = \frac{S_A + S_B}{N + m + \varepsilon}$$

7: **return** IoU

B. SLAM Details

We use the front right camera of Spot for SLAM visual inputs and Spot embedded IMU sensor for SLAM odometry input. We use the following ROS parameter for RTAB-Map configurations.

- `rtabmap_odom` module
  - `subscribe_rgb`: True
  - `subscribe_depth`: True
  - `approx_sync`: False
  - `wait_imu_to_init`: False
  - `publish_tf`: False
  - `publish_null_when_lost`: True
  - `Odom/Strategy`: 1
  - `Vis/CorType`: 0
- `rtabmap_slam` module
  - `subscribe_rgb`: True
  - `subscribe_depth`: True
  - `approx_sync`: True
  - `publish_tf`: True
  - `wait_imu_to_init`: False
  - `publish_tf`: False
  - `publish_null_when_lost`: True
  - `Grid/RayTracing`: True
  - `Grid/MaxGroundHeight`: 0
  - `Grid/MaxObstacleHeight`: 1
  - `Grid/FootprintLength`: 1.5
  - `Grid/FootprintWidth`: 1.0
  - `Grid/FootprintHeight`: 1.0
  - `GridGlobal/FootprintRadius`: 0.4

C. Graph Constructor Details

1) Object Detection: We detect the following objects during the experiment, using the threshold as 0.1.

```
["cabinet", "handle", "shoe", "can",
"toy", "cloth", "box", "open_box",
"bottle", "yellow_barrier", "table",
"mug", "fork", "plate", "drill", "banana"]
```

While this threshold is low and could introduce false positives, we keep task-related information during the detection stage. After the detection for each frame, we filter out false positives using multi-view consistency and temporal consistency.

After the object detection, we will post-process the object detection and filter out some detections. Specifically, the post-processing includes

- Normal estimation: we estimate the normal vector of each pixel by computing image gradient along x axis and y axis. By taking cross-product and normalizing, we could estimate the normal directions for each pixel. This information will be passed into object node for downstream manipulation.
- Voxel downsampling: we will downsample object point cloud to ensure efficient point cloud storage. Specially, we choose the downsample size as 0.02 m, except for door handles. Since door handles have a thin structure, downsampling them might result in a few points. Instead,

we will downsample door handles after finishing the association.

- Remove outliers: 3D point clouds might have outlier points due to noisy depth observations. We will compute k-nearest neighbors (kNNs) of each point and the average distance from each point to their kNNs. If the average distance of one point is abnormal, it will be regarded as the outlier and discarded. Our implementation is similar to Open3D's `remove_statistical_outlier`. The only difference is that we implement our method using PyTorch3D knn function to ensure efficiency. We choose `nb_neighbor`=100 and `std_ratio`=0.5.

2) Object Association: The association module links the fresh detections from the current frame to the persistent object nodes already stored in the scene graph. Doing so avoids creating duplicate nodes for the same physical object and keeps the graph consistent as the robot moves. Concretely, the object association consists of the following pipelines:

1) **Prepare point clouds.** Every existing graph node carries a raw point cloud. We down-sample each cloud with a 5 cm voxel grid so that later computations are fast and memory-friendly, except for objects with thin structures, such as door handles.
2) **Discard far-away candidates.** For each detection we compute the straight-line distance between its centroid and the centroid of every graph node. Only nodes that fall within a 0.40m radius of *any* detection are kept for further checks; the rest are ignored for this frame.
3) **Enforce label agreement.** We never merge objects of different classes (a mug can never be merged with a cabinet, for example). A simple label comparison removes those impossible pairs and enforce semantic consistency.
4) **Measure geometric overlap.** We calculate the point-cloud IoU between every remaining detection–node pair in one batched GPU call. The distance threshold used inside the IoU test is 5 cm for tiny objects and 10 cm otherwise.
5) **Accept or reject the match.** For each detection we keep the graph node with the highest IoU. A match is accepted if that IoU is at least 0.15.

After associating new detection with graph memory, we will update the graph according to the following criteria.

- **If a match is found.** The detection's point cloud is fused into the matched node, its centroid is updated with an exponential moving average, and the node's timestamp is refreshed.
- **If no match is found.** A brand-new node is created for the detection.
- **Multi-view consistency.** Nodes that are only visible for less than three frames are considered stale and invalid.

This purely geometric, label-aware strategy is robust to viewpoint changes, tolerant of partial occlusion, and runs in real time.

3) Voxel Space: Using the depth image and the camera intrinsics we generate a dense ray-bundle for the entire field of view at every frame. Each ray is sampled at regular 3 cm intervals out to the maximum sensor range, then transformed from the camera frame to the world frame and snapped to the nearest voxel in a fixed 3-D grid that spans the workspace.

For every voxel we assign one of four high-level labels that are used by the high-level planner:

- `unexplored`: the voxel has never been touched by any ray (the depth reading at the projected pixel is zero);
- `free`: the voxel is in front of the measured depth surface, which means the space between the camera and the first hit is known to be empty;
- `unknown`: the voxel is hit by a ray but lies *behind* the first visible surface, so it may or may not contain an object;
- `outside`: the voxel is beyond any wall plane, below the ground plane, or outside a user-defined bounding box of the room.

The computation process can be detailed as below:

1) **Transform and voxelise rays.** The pre-computed ray cloud is warped into world coordinates and voxelised in one GPU batch. This yields a list of candidate voxel indices for the current frame.
2) **Compare ray depth to image depth.** At each image pixel we look up the measured depth.
   - No valid depth reading → all voxels along that ray are labelled `unexplored`.
   - Ray point in front of the depth surface → `free`.
   - Ray point behind the depth surface → provisional `unknown`.
3) **Mark outside voxels.** Voxels that fall behind any wall plane, or below the floor, are overwritten with the label `outside`.
4) **Resolve unknown voxels.** The system tests voxels against the bounding boxes of known objects and utilize semantic information.
   - Inside a unexplored container (e.g. cabinet) → keep as `unknown`; its node ID is stored so the planner knows which object hides it.
   - Overlaps an existing object point cloud → mark as `occupied`.

Also, we have a specific order for label overwriting. For example, a voxel labelled as `unknown` cannot be `unexplored` again. Specifically, `outside` > `unknown` > `free` > `unexplored`.

This voxel map is very valuable for deciding the occlusion information and edge information. For instance, if many `unknown` voxels lie behind a closed cabinet door, we could know that the cabinet is not explored yet. if the unknown space is behind a pushable box, the VLM will suggest the `push` skill instead.

4) Relation Detection: Given such voxel representation, we can define the following object relations:

- `of`: If a child object can be possibly a part of the parent object, such as a handle is possibly a part of a cabinet, and their point cloud centroids are close enough, the child object node is `of` the parent object node.
- `inside`: If a child object is found after opening or flipping a parent object, the child object is `inside` the parent object node.

- `on`: If a child object's bounding box is within the parent object's bounding box in the x-y plane, and the child object's lowest z value is close to the parent object's highest z value, the child object is `on` the parent object node.
- `under`: If a child object is found after sitting down or lifting, the child object is `under` the parent object node.
- `behind`: If a child object is found after pushing a parent object aside, the child object node is `behind` the parent object node.

More concretely, there are two ways to create relations.

1) **Interaction–driven edges.**
Whenever the robot executes an interactive skill we memorize the last action information, including the manipulated object ID and action type. If a brand-new object appears in the very next frame, we assume it was hidden by the object that was just manipulated and create an edge:

Skill just performed	Relation assigned
`open`	`inside`
`lift` or `check_bottom`	`under`
`push`	`behind`
`flip`	`inside`

2) **Geometry–driven edges.**
After all nodes for the current frame have been processed we call `update_edges()`. This routine scans every pair of nodes and adds missing edges based on simple geometric tests on their bounding boxes and centroids:

a) **Part–whole (`of`) edges.**
- Only the class pair *handle* (child) → *cabinet* (parent) is considered at the moment.
- We calculate the 3-D box IoU between every handle and every cabinet. If the *inner* IoU (handle volume fully inside the cabinet volume) exceeds 0.50 we add an `of` edge.
- A handle can have *one and only one* cabinet parent.

b) **Support (`on`) edges.**
- For every pair of distinct boxes we check five axis-aligned conditions:
  - the lower object's top face is within 5 cm of the upper object's bottom face;
  - the lower object's footprint is fully contained in the upper object's footprint in both x and y, with a 5 cm margin;
  - no earlier parent–child link exists between this pair.
- When all conditions hold we add an `on` edge.

Algorithm 2 Serialize Graph

Input: A graph G with nodes V and edges E
Output: A string `texts` representing the serialized graph

```
Initialize texts ← "root\n"
Initialize to_visit ← empty stack
Add root_node to to_visit
while to_visit is not empty do
    // Append the text of the current node to final texts
    Pop first node curr_node from to_visit
    Obtain text curr_text of curr_node
    Append curr_text to return_texts

    // Find children nodes and add to to_visit
    child_nodes ← curr_node.get_children()
    for all child_node in child_nodes do
        Add child_node to to_visit
return return_texts
```

D. Task Planner Details

1) Graph Serialization: We serialize the graph using the depth-first search. The detailed algorithm is presented in Algorithm 2. We make sure that our graph will not form a cycle during the graph building process.

2) Prompts Examples: Here are all the prompt examples we use. We provide minimal prompt examples to the robot for task planning.

```
# explore one cabinet with one handle
graph:
root
\---cabinet_0
    \---handle_1 [obstruction]

# answer
action:
open handle_1 # affected_objects:
cabinet_0, handle_1

# explore one cabinet with multiple
handles
graph:
root
\---cabinet_0
    |---handle_2 [obstruction]
    \---handle_1 [obstruction]

# answer
action:
open handle_2 # affected_objects:
cabinet_0, handle_2
open handle_1 # affected_objects:
cabinet_0, handle_1

# explore one cabinet with multiple
handles, while some handles are opened
graph:
```

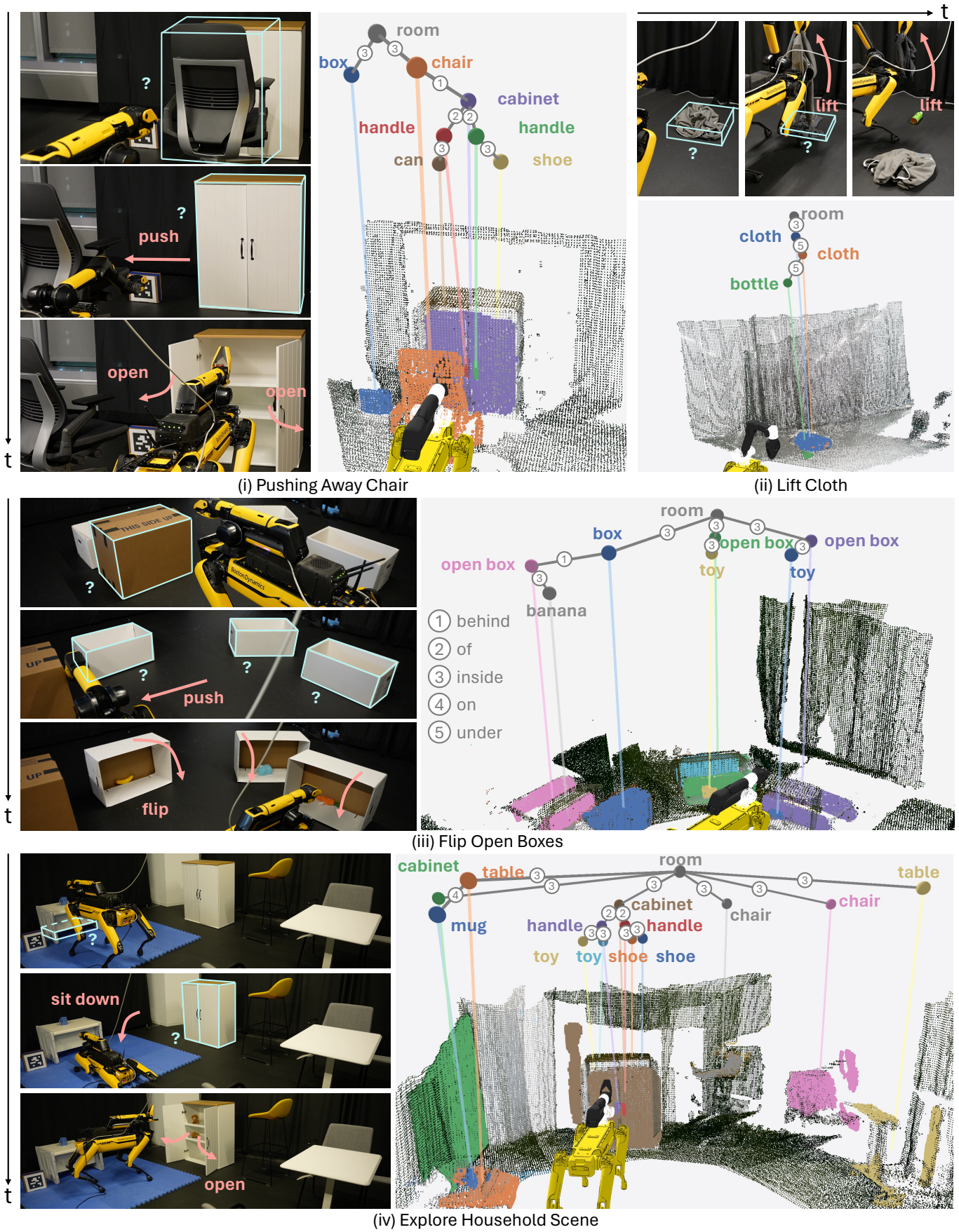

Fig. 1: **Qualitative Results.** We evaluate our system's exploration capabilities across various tasks, including pushing the chair aside to reveal space behind it, lifting cloth to check underneath, flipping open boxes to inspect the contents, and exploring a household scene. These tasks showcase the system's ability to generalize across different object types, scenarios, and object relations. Additional tasks can be found on the project page.

```
root
|---chair_3 [obstruction]
|---box_4 [obstruction]
|---cloth_5 [obstruction]
\---cabinet_0
    |---handle_1 (opened)
    \---handle_2 [obstruction]

# answer
action:
push chair_3 # affected_objects: chair_3
push box_4 # affected_objects: box_4
lift cloth_5 # affected_objects: cloth_5
open handle_2 # affected_objects:
cabinet_0, handle_2

# explore several cabinets and obstruction
objects
graph:
root
|---cabinet_0
|   |---handle_1 (opened)
|   |   \---object_3
|   \---handle_2 (opened)
|       \---object_5
|---object_7
|---box_9 [obstruction]
\---cabinet_6
    \---handle_4 [obstruction]

# answer
action:
open handle_4 # affected_objects:
cabinet_6, handle_4
push box_9 # affected_objects: box_9

# explore several cabinets and obstruction
objects
graph:
root
|---cabinet_0
|   |---handle_1 (opened)
|   |   \---object_3 (moved)
|   |       \---object_8
|   \---handle_2 (opened)
|       \---object_5 (moved)
|---object_7
|---box_9
\---cabinet_6
    \---handle_4 (opened)

# answer
action:
none

# explore several cabinets and obstruction
objects
graph:
root
|---cabinet_0
|   |---handle_1 (opened)
|   |   \---object_3 (moved)
|   |       \---object_9 (moved)
|   \---handle_2 (opened)
|       \---object_5 (moved)
|---object_7
\---cabinet_6
    \---handle_4 (opened)

# answer
action:
none

# explore several cabinets and obstruction
objects
graph:
root
|---sealed_box_1
|---sealed_box_2
\---open_box_3 [obstruction]

# answer
action:
flip open_box_3 # affected_objects:
open_box_3
```

E. Low-Level Skills Details

All skills are executed with Spot's "vision" or "body" frame. Base velocity for walking is capped at 0.10 m/s for safety reason. Each actions are detailed as below.

1) `open`:

- **Move.** The robot first navigates to a pose 1.20 m in front of the cabinet, facing its negative normal direction. Then we will predict the grasping pose aligning with the handle, according to handle normal direction and door normal direction.
- **Grasping.** The robot tries to approach the handle and close the gripper. If the gripper is fully closed, that means the grasping fails due to potential perception uncertainty. If the grasping fails, the robot will re-try the grasping with different action parameters.
- **Pull Back.** The robot pulls 15 cm straight back. Since we will use impedance controller for this task, the end effector will be compliant and move according to the door articulation structure. Therefore, we can adjust the robot motion and continue to open the door.
- **Reset.** The robot opens gripper, stows arm, and backs away 20 cm.

2) `lift`:

- **Move.** The robot stops one meter in front of the target object, aligning the wrist vertically above the center point.
- **Pick.** The arm descends to a waypoint 20 cm above the object, then 5 cm above the surface, and closes the gripper.

- **Place.** The lifted item is raised another 20 cm and placed aside.
- **Reset.** The base steps back 80 cm, pauses ten seconds for perception and updating the graph, then returns to its original position.

3) `push`:

- **Move.** The robot positions itself one meter from the obstacle with the gripper centered on the mid-height of the object.
- **Offset.** Move aside to push objects away.

4) `sit`:

- **Lower.** A single command sends Spot into its built-in "sit" posture.
- **Hold.** The robot remains seated, keeps the arm stowed, and updates the scene graph.

5) `collect`:

- **Move.** The robot moves in front of objects according to their normal directions.
- **Grasp.** The grasp module cycles through lateral offsets of 0 cm and ± 5 cm and vertical offsets of 0 cm and ± 10 cm until confirming a firm grasp.
- **Retreat.** After closing the gripper the base reverses 20 cm to avoid the collision with environments.
- **Place.** The object is deposited into a basket on the robot back through fixed waypoints.
- **Reset.** The arm is stowed, the gripper is closed, and the robot returns to a standing posture, ready for the next command.

F. Exploration Ending

When there is no unknown or obstruction object node within the environment, we will stop the exploration. The maximum depth of the graph in our experiment setting is 5.

G. Clarification on Non-Interactive Actions

For the non-interactive actions, such as `move to` and `sit`, they are unified with interactive actions, such as `open`. Because our actionable graph representation encode geometric information, semantic information, and affordance information, all non-interactive actions and interactive actions can be unified in our framework.

II. Experiments

A. Full Qualitative Results

Due to the page limitation, we only show partial results in our main paper. Here we list more complete qualitative results in Figure 1.